

1. Evidence (label each clearly)

- The centralised get-log-events read-back (Task 2).

```
(melon@melon)-[~]
$ TS=$(date +%s000)
while IFS= read -r line; do
aws $EP logs put-log-events --log-group-name /ccse/app --log-stream-name auth \
--log-events timestamp=$TS,message="$line" >/dev/null; TS=$((TS+1000)); done < auth.log

# Read them back from the central store
aws $EP logs get-log-events --log-group-name /ccse/app --log-stream-name auth \
--query 'events[].message' --output text

2025-03-01T09:00:01 LOGIN_OK user=ahmad ip=10.0.0.5      2025-03-01T09:01:10 LOGIN_FAIL user=admin ip=203.0.11
3.9      2025-03-01T09:01:12 LOGIN_FAIL user=admin ip=203.0.113.9      2025-03-01T09:01:15 LOGIN_FAIL user=a
dmin ip=203.0.113.9      2025-03-01T09:01:18 LOGIN_FAIL user=admin ip=203.0.113.9      2025-03-01T09:01:22 L
OGIN_OK user=admin ip=203.0.113.9      2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB
```

- The failed-login count grouped by IP (Task 3).

```
(melon@melon)-[~]
$ # How many failed logins, and from which IP?
grep LOGIN_FAIL auth.log | awk '{print $4, $5}' | sort | uniq -c

# Distinguish a log (durable record) from an event (a trigger):
# an EVENT would be 'alert: 4 failures from 203.0.113.9' fired in near real time.

4 ip=203.0.113.9
```

- The hash-chained log and proof that tampering changes the final hash (Task 4).

```
(melon@melon)-[~]
$ >....
TAMPER=$(tail -1 auth.tampered.chain | awk -F' \\| ' '{print $2}')

echo "4. Final Hash Comparison:"
echo "    Original: $ORIG"
echo "    Tampered: $TAMPER"
echo ""

# Verify if tampering detected
if [ "$ORIG" != "$TAMPER" ]; then
    echo "❌ TAMPERING DETECTED!"
    echo "    The hash chain proves the log was altered."
    echo ""
    echo "    📋 Evidence:"
    echo "    Original entry: $(grep EXPORT_DATA auth.log)"
    echo "    Tampered entry: $(grep EXPORT_DATA auth.tampered)"
    echo ""
    echo "    🕒 The final hash changed, proving tampering!"
else
    echo "✅ No tampering detected"
fi
EOF

# Make it executable
chmod +x check_tamper.sh

# Run it
./check_tamper.sh
4. Final Hash Comparison:
Original: ababa787b4bf524d9daddca8c48e4909fc105769a6f17574f42cefe8f81233cf
Tampered: 72f1d53774a3a938fa7bd3a88f67894e5a64055a41ee7511eac53d7bd89d859b

❌ TAMPERING DETECTED!
The hash chain proves the log was altered.

📋 Evidence:
Original entry: 2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=500MB
Tampered entry: 2025-03-01T09:01:40 EXPORT_DATA user=admin ip=203.0.113.9 size=5MB

🕒 The final hash changed, proving tampering!
```

- The correlation ALERT output (Task 5).

```
(melon@melon)-[~]
$ IP=203.0.113.9
FAILS=$(grep -c "LOGIN_FAIL.*$IP" auth.log)
SUCCESS=$(grep -c "LOGIN_OK.*$IP" auth.log)
EXPORT=$(grep -c "EXPORT_DATA.*$IP" auth.log)
echo "IP=$IP fails=$FAILS success=$SUCCESS export=$EXPORT"

if [ "$FAILS" -ge 3 ] && [ "$SUCCESS" -ge 1 ] && [ "$EXPORT" -ge 1 ]; then
    echo 'ALERT: probable brute-force → compromise → data exfiltration';
fi

IP=203.0.113.9 fails=4 success=1 export=1
ALERT: probable brute-force → compromise → data exfiltration
```

- The containment rule and the evidence hash file (Task 6).

```
(melon@melon)-[~]
$ # Option 1: Run the command without pipe (show all rules)
docker run --rm --cap-add=NET_ADMIN alpine sh -c \
'apk add -q iptables 2>/dev/null; iptables -A INPUT -s 203.0.113.9 -j DROP; iptables -L INPUT -n'

# Option 2: Use sh -c with proper quoting for pipe
docker run --rm --cap-add=NET_ADMIN alpine sh -c "
apk add -q iptables 2>/dev/null
iptables -A INPUT -s 203.0.113.9 -j DROP
iptables -L INPUT -n | tail -3
"

Chain INPUT (policy ACCEPT)
target    prot opt source                destination
DROP      all  --  203.0.113.9            0.0.0.0/0
Chain INPUT (policy ACCEPT)
target    prot opt source                destination
DROP      all  --  203.0.113.9            0.0.0.0/0
```